Journal of Systems Architecture 168 (2025) 103532

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Live migration of compiled Wasm modules across the Compute Continuum

E. Tinto *, L. Marchiori, T. Vardanega

Department of Mathematics, University of Padua, Italy

ARTICLE INFO

Keywords:
WebAssembly
Compute Continuum
Embedded systems
Live migration

ABSTRACT

The compute continuum is a model of deployment and computation that envelopes the Cloud and the Edge into a seamless runtime infrastructure. Vast heterogeneity at the Edge is one of the main challenges of the continuum. A lightweight uniform runtime, such as the one offered by WebAssembly (Wasm) may be an apt response to that. The other main challenge is the occasional need for computations to relocate. This happens when local resources are insufficient or inadequate to meet the application requirements or when location changes in the physical world require the computation to move with them. The notion of *live* migration applied to Wasm computations is not new. Yet, state-of-the-art solutions focus on either non-standard Wasm runtimes or interpreted modules. Supporting live migration for *compiled* modules across heterogeneous nodes is still an open challenge. This paper presents a mechanism to meet that need. By injecting checkpoint and restore procedures into a function bytecode within a Wasm module, we enable it to save its execution state and resume from it after a migration event. This paper presents two strategies for that, one of which with notably small run-time overhead. To assess the quality of the proposed strategies, we performed an empirical evaluation based on an open-source benchmark. The tools developed in this work are entirely open-source.

1. Introduction

1.1. The Compute Continuum

The availability of computing resources and extended connectivity have led to the proliferation of (possibly) small connected computing devices in the periphery of the network. While often resource-constrained, the reduced latency that those devices can offer, compared to their kins hosted in the Cloud or datacenters, is an attractive asset. The concept that has recently emerged in that regard is a heterogeneous yet unified infrastructure that extends from Edge devices to Cloud datacenters, also known as the *Compute Continuum*. As noted in [1], heterogeneity is one of the main challenges of the Continuum, which is massively evident at the Edge of the network. Heterogeneity affects both the hardware and software strata, making the development of end-to-end analysis models extremely difficult and encouraging *silo*-based approaches [2]. However, the prospect of achieving more elastic and efficient systems makes this strand of research particularly worthwhile, as noted by [3]. Another important challenge, tied with computing at the Edge, is the occasional need for computations to relocate. This need arises in at least two circumstances: when local resources become insufficient or inadequate for the application to achieve its goals or when displacements in the physical world require the computation to move accordingly.

Two technologies that bear promises of interest to the compute continuum have emerged from a field that seems quite far from it: WebAssembly and Rust.

WebAssembly is a bytecode format designed for portability and isolation, initially intended as a faster alternative to JavaScript engines for web browsers. The WebAssembly specification (Wasm, for short) requires that each module, the building block of applications, be executed in a sandboxed environment. System interfaces and memory addresses are accessed under strict (and statically checked) rules. WebAssembly helps to circumvent heterogeneity in the Continuum, cf. [1]. Several works, e.g., [4–6], and [7], have vouched for the potential of this technology for embedded systems.

Rust is a programming language that provides for statically-checked memory safety. Its relevance to real-time and safety-critical systems, echoed by [8,9], is becoming increasingly apparent, as testified by efforts like Ferrous Systems' [10], whose Rust compiler toolchain has attained safety- and mission-critical certifications. Interestingly, Rust has been used for the development of Wasmtime [11], one of the best-known Wasm runtimes.

It is our contention that Wasm's portability and Rust's memory safety can jointly help address the heterogeneity challenge of the Continuum, providing a uniform and memory-safe execution layer across devices.

* Corresponding author.
E-mail addresses: edoardo.tinto@phd.unipd.it (E. Tinto), luca.marchiori@hotmail.it (L. Marchiori), tullio.vardanega@unipd.it (T. Vardanega).

https://doi.org/10.1016/j.sysarc.2025.103532
Received 6 February 2025; Received in revised form 2 July 2025; Accepted 26 July 2025
Available online 6 August 2025

Traditionally, embedded and real-time applications are tightly coupled with their hardware layer. The system model that we discuss in this paper seeks to separate the hardware layer from the software application, much like the Cloud does, but in a manner that stays sensitive to time. In the Cloud, compute resources are federated into *pools*, with the accrued benefit that applications may, in principle, *move* opportunistically where execution becomes more convenient for the infrastructure, for the application, or for both. A portable execution environment, sufficiently small-scaled to be hosted in resource-constrained devices, becomes handy to fulfill our vision of the compute continuum.

In addition to an environment of this sort, isolated from the host and portable, applications intended as aggregates of coordinated computations might benefit from the dynamic redeployment of components. This redeployment, which must ensure the seamless continuation of the computation, is called *live* migration. While close to the notion of offloading, well known in Cloud and Edge systems, it has some important differences. The principal trigger of an offloading procedure is a fault in the sending node, which becomes unable to complete a given computation. The computation is then redeployed on a node with more available resources at that time. Instead, in the envisioned model, computations migrate for *convenience*, opportunistically, in the quest to optimize user-defined metrics. For this kind of mobility to be effective, it has to be inexpensive for resource requirements and faster for execution than offloading. Moreover, it has to allow the decision of when and where to migrate to be taken in a timely fashion.

This work presents a novel approach to support the live migration of *compiled* Wasm computations by partitioning target functions within a Wasm module into regions, and injecting run-time procedures in them to perform regular checkpoints at region boundaries. Working entirely at the bytecode level, this proposal could be applied as-is on any Wasm runtime, even those equipped with interpreters.

1.2. Related work

The most discussed approach to realize live migration of application components is to use containers and a Linux-native toolset named CRIU [12]. Checkpoint and Restore In Userspace (CRIU) can stop and resume OS processes, producing a snapshot of the process state. Focusing specifically on Cloud systems, the authors of [13] present a CRIU-based approach for live migration across hosts with identical architectures. Instead, the authors of [14] propose a cross-architecture solution. [15] considers the case of latency-critical applications. The authors of [16] try to improve the efficiency of live migration for containers. Interestingly, [17] explores the possibilities of migrating a Wasm computation in a similar manner, across different Wasm runtimes. All these works are based on CRIU (cf. Section 2.4).

Relatively few works have addressed the topic of live migration in the context of WebAssembly. [18] presents a mechanism to perform live migration, where the computation is intended as an HTML5 *mobile web worker*, and whose state is, in fact, the state of the web worker, serialized for portability across the network. In [19], the authors introduce a mechanism for live migration of interpreted Wasm components, which addresses the need for cross-architecture migration at the cost of slower speed of execution. However, support for Wasi, Wasm standard set of API [20], is not discussed. Another solution targeting interpreted components is presented in [21], which also integrates the use of a trusted execution environment (TEE) in their proposal and experimental support for Wasi. [22] suggests an approach to migrate idle components, thus not actively engaged in a function call, thereby preserving the state of those components. Finally, with a focus on embedded and resource-constrained devices, the authors of [23] mitigate the security risk arising from the adoption of a multi-tenant infrastructure, by enabling the migration of Wasm computations. An interesting project is presented in [24], to achieve the live migration of Wasm modules using a tailored Wasm runtime, which makes it less attractive. In contrast to these works, our solution enables live migration for compiled Wasm modules through the instrumentation of standard Wasm bytecode, which makes it quite unique. Worthy of mention in this regard is the open-source project Wanco [25], a compiler supporting checkpoint and restore of Wasm functions across POSIX-compliant heterogeneous architectures. To the best of our knowledge, supporting live migration for compiled modules, including those actively involved in a call, is still an open question. The remainder of this work shows how this can be done.

1.3. Contributions

In this work, we aim to advance the research in the fields of time-predictable Compute Continuum with three contributions:

- A system design leveraging WebAssembly (Wasm) to decouple the infrastructure from the application, and enriched with a checkpoint and restore mechanism for computations running as functions within a Wasm module. This mechanism is a requirement for *live migration*. We introduce two checkpoint strategies, presented in Section 3, one of which introduces negligible overhead for non-migrating computations. Section 2 provides both the background to and a description of the envisioned model.
- We introduce a software instrument capable of injecting run-time procedures into an existing Wasm module, which allows performing occasional checkpoint and restore procedures. The proposed approach paves the way for other and more efficient strategies. Moreover, the proposed approach enables live migration of *compiled* components and can be applied, *as-is*, to any Wasm runtime, thus granting maximal flexibility. In Section 3 we provide a description of the injection procedure.
- We provide a preliminary evaluation of the run-time penalty incurred by components undergoing periodic checkpoints, using an established scientific benchmark. From these results, discussed in Section 4, we reflect further on the cost of checkpointing, as a function of size and frequency.

The remainder of this paper is organized as follows. Section 2 offers an introduction to the envisioned model, comparing it to state-of-the-art solutions and a rationale for live migration. Section 3 describes the proposed approach to support checkpoint and restore in Wasm, while Section 4 describes the methodology of our empirical evaluation and its result. Section 5 reasons on the limitations and the possible extensions of this work. Finally, Section 6 presents our conclusions and future research directions.

2. Background on the Compute Continuum

2.1. The envisioned model

The infrastructure on top of which modern software systems execute is becoming increasingly heterogeneous. As Cloud and datacenters offer abundant and cheap computing resources, usually rather distant from where data are generated, the periphery of the network, the Edge, is populated by an increasing number of potentially small devices with limited computational power but close to where data actually reside. More than close, those devices are often *embedded* in cyber–physical systems, sometimes performing sensing and actuation tasks. Two major trends have emerged recently. First, increasing computing power available at the Edge, facilitated by cheaper hardware appliances and ubiquitous connectivity. Its main outcome is bringing forward the Edge as a suitable destination for hosting compute tasks. Second, the integration of hardware accelerators in those cyber–physical systems, often in the form of graphic processing units (GPUs), even under firm, if not yet hard real-time requirements, as in [26,27], and [28]. Those trends amplify the heterogeneity of the Edge segment of the network.

The common approach to tackle this heterogeneity, in industry or academia, is to integrate those heterogeneous technologies under

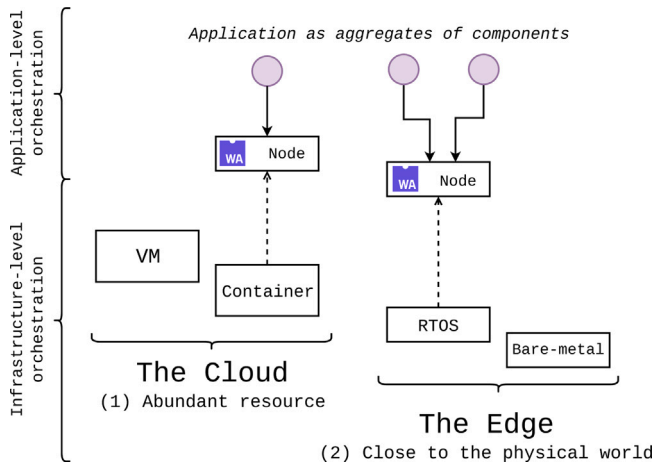


Fig. 1. The envisioned model for the Compute Continuum. Orchestration happens at the infrastructure level and at the application level.

the aegis of some software framework. Doing so allows abstracting the application logic from the underlying complexity. While appealing in some contexts, e.g., scientific computing, it gives rise to systems so complex to analyze in the presence of real-time requirements. A different approach to overcome heterogeneity consists of instrumenting each node, be it an Edge device or a virtual machine/container from a Cloud provider, with the same execution environment. This would allow the same application component to be scheduled for execution in any node in the system that matches its import interfaces and resource requirements.

There are some advantages to the latter approach. It encourages the idea of an infrastructure encompassing both the Cloud and the Edge, where computing resources are accessed as a single *pool*, similar to what already happens in Cloud systems. The infrastructure so intended could host different applications, thus severing the ties between a piece of hardware and a specific software component, as partially mentioned here [23]. Having each node expose the same runtime environment is a key requirement to support the live migration, hence stateful and connection-preserving, of the computations. Furthermore, the envisioned model restrains from imposing any architecture to the application, e.g., functional or stateless, leaving that as an application-level decision.

Yet, in a dynamic context where some devices, especially those at the Edge, might disconnect or change their geographical locations, coordination among application components becomes crucial. In other words, fragmenting an application into components requires orchestration. Moreover, orchestration decisions should happen in a timely manner. Orchestration should happen at two places: (1) at the infrastructure level, to aggregate and assign resources to applications; and (2) at the application level, to schedule the components over the allotted resources. Fig. 1 offers a graphical illustration of this two-level orchestration strategy where orchestration occurs at two levels: (1) the infrastructural level, when a federation of nodes is assigned to an application; and (2) the application level, when the coordination of components takes place. An analysis of the temporal behavior of an application could not ignore the (application-level) orchestration strategy of its components, and such an analysis requires reservation and isolation guarantees at the infrastructure level.

2.2. Key technologies

A promising technology for the role of a lightweight and portable runtime environment is WebAssembly [29]. Although the idea of portable bytecode is not new, and Java is proof of it, WebAssembly has a unique flavor. It is considerably lighter than a whole JVM, offers a minimal set of functionalities, easing the standardization process, and regulates the interactions with the "outside", or host, environment clearly, as in [30]. A function defined in a Wasm component executes in a sandbox, which means accessing only a statically defined set of imports from the host and specific portions of continuous memory, referred to as linear memories. Therefore, the import requirements for instantiating and running a Wasm computation can be determined entirely by its declared imports and memory. The potential of this technology, both for the Continuum and for embedded systems, has already been discussed in Section 1.1.

There exist different Wasm runtimes, but our choice stands with Wasmtime [11], for two reasons. It is developed in Rust, a programming language that provides memory safety, with minimal use of `unsafe` regions, hence unchecked by the static analyzer and (as the name suggests) not so safe. Moreover, according to the developer team [31], there are growing efforts to formally verify this runtime and its integrated compiler. Wasmtime embeds a just-in-time (JIT) compiler, always Rust-written, named Cranelift, designed for fast compilation and capable of producing compiled code that can almost match the LLVM-compiled one at a fraction of the latter compile-time cost, according to [32]. While, for now, Cranelift backend targets just a handful of architectures, an interpreter, Pulley, is available with fewer requirements on the target architectures.

Executing Wasm modules at the Edge raises the question of how to accommodate access to sensors and actuators. In other words, how to interact with the hardware interfaces that give access to peripheral devices. To date, two approaches have been proposed to this end. One solution, supported in [33] with promising results, uses the Wasi component model to split device drivers into two components, the first running as a Wasm component and the other developed directly in the hosting node. A Wit – Wasm Interface Type, a language used to define the interfaces of a Wasm component – interface regulates the interaction between the two pieces of software. The other solution, cf. [34], proposes a method for Wasm modules to handle hardware interrupts directly.

2.3. On the use of containers

Using containers as building blocks for applications is a well-established approach in previous research on the Continuum. The fact that mature containerization technologies exist, e.g., Docker, and virtually all Cloud-native orchestrators leverage containers, are two of the main driving factors supporting this approach. However, when developing a model that also encompasses the edge of the network, that route shows two limitations. The first is discussed here, and the second is discussed in Section 2.4. Containers, though much smaller than virtual machines, are still relatively large in terms of memory and resource footprint for Edge devices. The deployment of a container also takes longer compared to the instantiation of a Wasm component, as seen in a test-case scenario using Oakestra, first presented in [35], an application-level orchestrator. Tables 1 and 2 report our findings from preliminary experiments on a dedicated server mounting an Asrock B450 Steel Legend board (AMD64) running Kubuntu 24.04, Wasmtime v27.0.0, and Docker v26.1. Our results show that instantiating the same computation as Wasm bytecode is less time consuming than its deployment in a Linux container based on Docker. The memory footprint in the case of a Wasm computation is also smaller than its corresponding containerized counterpart, as shown in Table 3. However, it should be mentioned that the size of the Wasm bytecode can be further contained. For example, compiling without dependencies from a standard library (as in `no_std` for Rust) might produce smaller compiled artifacts.

However, containers stand up to Wasm computations in terms of execution performance. Table 4 reports the execution times of four benchmarks measured for the same computation running as Wasm bytecode over Wasmtime and as a native executable within a Docker

Table 1
Run-time cost of instantiating a Wasm module in JIT manner. Compilation is the most time-consuming step.

Step	Duration (ms)	Cumulative time (ms)
Initialization	6.967	6.967
Compilation	322.474	329.441
Linking	0.068	329.509
Instantiation	0.589	330.098
Function Call	0.020	330.118
Total	330.118	

Table 2
Run-time cost of deploying a Linux Alpine Docker container. Creating the container is the most time-consuming step.

Step	Duration (ms)	Cumulative time (ms)
Prepare OCI specs	0.090	0.090
Create container	462.860	462.950
Setup task	49.465	512.415
Attach network	29.380	541.795
Start task	8.632	550.427
Total	550.427	

Table 3
The memory footprint of the computing bundle, as a Docker container and a Wasm module. In general, Wasm is more lightweight.

Application	Docker image size	Wasm module size
Benchmark app	9.87 MB	2.32 MB

Table 4
Execution times measured from three benchmarks (coming from PolyBenchC) while running as Wasm computations and within a container.

Benchmark	Wasmtime (s)	Container (s)
2mm	4.205396	1.053942
3mm	7.006338	1.740273
atax	0.014553	0.003233
floyd-warshall	43.996459	11.776329

container on Linux Alpine. The containerized version achieves better performance, in line with the findings of other works [36]. The reason for this performance difference may be better optimization in C-compiled binaries than in their Wasm-compiled correspondents. A key insight from this experiment is that applications might incorporate both containers and Wasm computations, knowing their respective strength points, to better accommodate dynamic workloads. An orchestrator capable of scheduling migrating Wasm computation and relatively static containers would be needed to this end.

2.4. Live migration

In the Continuum, migration is an opportunity if it preserves the execution progress, instead of reexecuting the computation entirely (as is the case for offloading). In the Continuum, proactively migrating a computation for optimization can improve metrics such as latency or energy usage. However, to be beneficial, live migration must be fast and minimally disruptive. Otherwise, it would be outperformed by a simpler restart. Another reason for migrating computations is to reduce contention for shared hardware resources, such as accelerators. Components needing access to those devices only for a brief portion of their execution time might be temporarily migrated to a node with GPUs, and then resume their computation elsewhere.

Questions might arise on other approaches to the same problem, for example, with regard to a solution based on containers or unikernels. Both of these technologies suffer from a similar impediment, which is how to preserve the state of the computation when migrating across two nodes, potentially with different architectures. Running as OS processes, the execution context, e.g., registers and system APIs, changes significantly between architectures, making the resume operation a very arduous task. While some research efforts attempted to do so, as in [37], its support in a state-of-the-art container technology such as Docker is presently still experimental. Moreover, containers and unikernels have other software components to bring along in a migration (such as some system libraries, for unikernels). Wasm has none, instead, which incurs a much leaner memory footprint than the two former technologies.

WebAssembly offers a more flexible environment for *live* migration. The architectural requirements for executing Wasm are few, among which can be found support for IEEE 754-2008 32-bit and 64-bit floating point and little endianness, as reported in [38]. The same Wasm bytecode can then be compiled (JIT) and run in any node exposing a WebAssembly runtime, that is, for the remainder of this paper, Wasmtime. Interestingly, *any* Wasi-compliant runtime would serve that purpose. Hence, there is no reason to require that each node should host the same Wasm runtime. For these reasons, we selected Wasm to realize our migration mechanism, as described next.

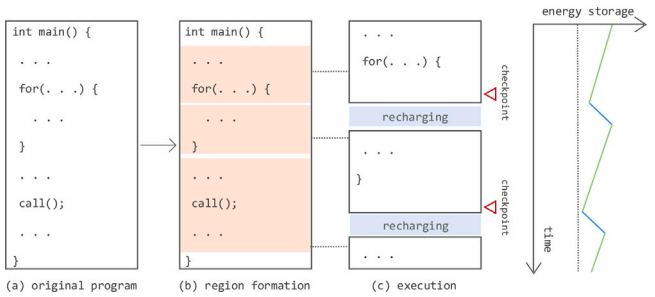


Fig. 2. Region formation and execution in EHS. Steps (a) and (b) happen at compile-time. The figure is a re-elaboration of the ones presented in [41].

3. Design of the solution

3.1. A glance at intermittent computing

The problem of advancing a computation in the presence of frequent interruptions, each requiring the checkpoint and restore of the system state, is a familiar topic in the domain of intermittent computing (IC) systems, more formally presented in [39]. For instance, in the context of energy harvesting systems (EHS). Those systems are used in on-board computers and scientific instruments for satellites. Their ability to run on solar energy without links to the power grid has also made them useful for on-earth applications, such as sensors and monitoring systems for agriculture. There, computations run on batteryless devices where, upon low-energy phases, the execution is interrupted as the devices turn off. In these systems, energy is 'harvested' from the surrounding environment, and the devices enjoy only very ephemeral energy storage in the form of physical capacitors.

A typical problem that energy harvesting systems face is granting forward progress, that is, the ability to execute a region between two adjacent checkpoints without being stuck to partially reexecute it forever, cf. [39]. Forward progress is usually achieved by partitioning the program into regions. At the end of each region, the state of the computation is stored in nonvolatile memory (NVM). The state is represented as registers and the stack. As shown by the experimental evaluation in [40], the cost of frequent checkpoints could be severe. Fig. 2 provides a graphical representation of this approach, inspired by illustrations in [41].

A promising approach to checkpoint and restore can be found in [41]. While investigating a solution to contrast the stagnation problem in EHS, the authors propose a *distributed* checkpointing mechanism. The motivation for this approach comes from the observation that many of the registers saved in memory during a checkpoint, the whole set

of *live-in* registers holding live values for the upcoming region, are unnecessary. In fact, only a subset of those registers will be accessed in the current region, while many will be read later on. Therefore, the authors propose a more efficient take on checkpoint and restore, limiting the checkpoint to the updated registers within the current region, corresponding to those defined there and the so-called *live-out* registers, meaning those registers that will be accessed by a successor region (in other words, registers that are *live-in* for some successive region). We repurposed the concept of partitioning execution into resumable regions to construct our migration solution.

3.2. Differences between wasm and energy harvesting systems

With the idea of leveraging distributed checkpoints to enable a Wasm computation to migrate across heterogeneous hosts, we should first consider some important differences between the scenarios of EHS and that of the Continuum. While the former considers applications as traditionally understood in embedded systems, tightly coupled with the underneath hardware, and statically deployed on a computing node, the latter adopts a more dynamic model. For instance, deciding the width of a region between checkpoints in the Continuum is considerably more difficult than doing it in an energy-harvesting application oblivious of migration. Different hosts might, in fact, have different optimal region widths.

The assumptions on energy availability also mark a difference between the two contexts. In EHS, the occurrence of low-energy phases is frequent, while in the Continuum, this might not be the case, so much so that the presence of non-volatile memory is not the norm.

Another important difference is that Wasm applications are organized in modules, each containing a list of functions that can be host-imported, defined locally, and possibly exported back into the host environment. The distributed checkpointing procedures should, therefore, be applied at the function level. For this reason, from now on, we should consider a single function as a target. The case of checkpointing multiple functions is discussed as well after introducing the basic algorithm for the single-function case.

Finally, in EHS, the region creation and the checkpoint insertion happen at compile-time. The state of the computation is expressed as execution stack and registers. This approach works when deployment is static, and migration never occurs. However, when these assumptions fall, it becomes a liability due to the difficulty of restoring both stack and registers on heterogeneous architectures. The case of OS processes migration is an example of this (see Section 2.4). In this work, we propose to inject the checkpoint and restore procedures directly into the bytecode. As a result, registers are not the main entities composing the component state, but variables are. Moreover, in the current and exploratory implementation, we perform checkpoints when the operand stack is empty. A proposal to achieve stack migration in Wasm is discussed in Section 5.

3.3. Injecting checkpoint and restore into a Wasm module

This operation is performed once, ahead of execution time. Doing so has at least two advantages. It leaves room for optimization without time constraints. A solution embedded in Wasmtime would have to be mindful of the requirement for fast compilation, which is a distinctive trait of this runtime. Moving this step before the JIT compilation, therefore, remove this issue entirely. Moreover, injecting the runtime procedures directly on the Wasm bytecode allows the same bytecode to run in any Wasm runtime, assuming that dependency (its imports) are satisfied. As a result, the solution described here could be applied to other runtimes and even to interpreting ones. The remainder of this section describes the injection mechanism.

As in the proposal of [41], the initial step consists of partitioning the selected function into regions. The approach chosen in this work is rather simple and consists of constructing a graph from the blocks in the function bytecode. Each non-nested, hence outermost block, becomes a region, as shown in Fig. 3. As a result, in the current implementation, checkpoints occur only when the value stack is empty.

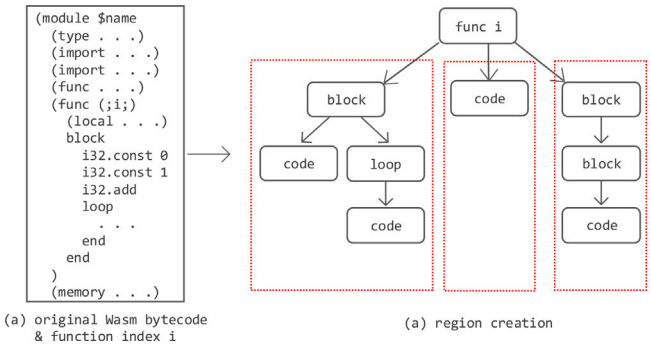


Fig. 3. The partitioning of a Wasm function into regions. Note that, thanks to the well-structured property of Wasm bytecode, all blocks are well-nested, and branching is possible only between a node and one of its predecessors (any outer nodes).

The second step concerns the production of a set of *live* variables to be stored during a checkpoint procedure. It is worth briefly mentioning how variables are organized in a Wasm module. Variables can be locals or globals (`globals`), and both can be mutable, whereas local variables only are divided into parameters (`params`) and function-local variables (`locals`). While constructing our checkpoint, we considered mutable `globals`, `params`, and `locals`. Due to the simple partitioning strategy adopted, the resulting regions are sequential, and the data flow is deterministic. As a result, at the end of each region, only the variables modified since the last checkpoint are saved in memory. We denote this set as $V_M(r_i)$, where r_i refers to the ith region of the function body. After producing the sets corresponding to each region, a mapping between each variable and a memory address is computed. These addresses will be used to store the selected variables during a checkpoint and to retrieve them upon resuming. Notably, each variable has only one associated address.

WebAssembly modules interact with the host memory through the use of *linear* memories. These are continuous address spaces, accessed through offsets, statically declared in the module bytecode, whose sizes are also statically defined (though dynamic changes are allowed). A module can access multiple linear memories. The Wasm runtime will take care of allocating these memory portions with respect to the description provided within the module. It is necessary that the host runtime supports dynamic memory allocation (e.g., via the Rust `alloc` profile). In the proposed approach, each function checkpoint is stored in a dedicated linear memory, added to the bytecode memory section during the injection of the checkpoint and restore procedures. Doing so prevents a faulty or malicious application from altering the state of its own checkpoint.

After the extraction of a region-specific set of altered variables and a mapping from these variables to addresses in linear memory, three runtime procedures are inserted at different stages.

1. An initial procedure loads from memory the index of the region from which the computation should resume. To this end, an `i32` variable is added to the `locals` set and updated at the completion of each region. This procedure is invoked once after instantiating the function and before executing the function body, as illustrated in Fig. 4.
2. As depicted in Fig. 5, at the entrance of each region r_i, a procedure checks whether we are in one of the following three states, acting accordingly:

   (a) The computation is resuming, but from a later region. In this case, each variable in $V_M(r_i)$ is restored from memory, and the execution jumps to the end of the region.

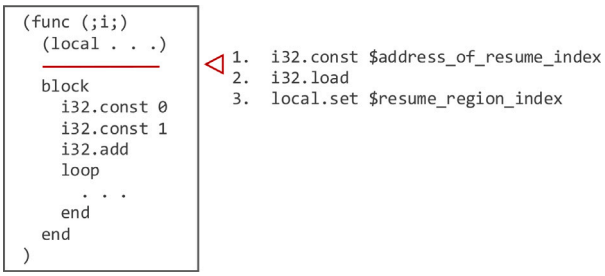


Fig. 4. The initial runtime procedure, loading from memory the index of the region from which resuming the computation.

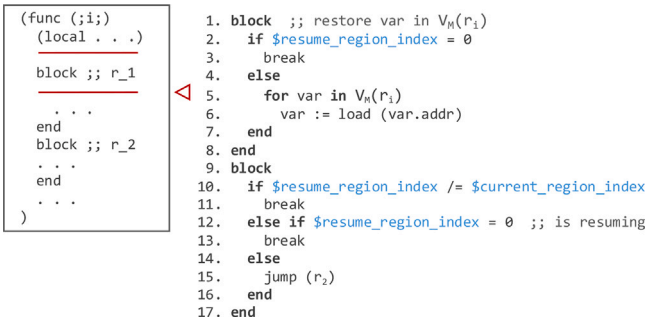


Fig. 5. The second runtime procedure, deciding whether or not to restore the variables in $V_M(r_i)$ and jump to the end of the region. For ease of understanding, the figure provides a portion of pseudo-code illustrating the behavior of the procedure, instead of the textual representation of the corresponding Wasm bytecode.

(b) The computation is resuming from this very region. Then the execution continues without resuming any variables.
(c) The computation is not resuming. The computation continues as in the previous case.

3. At the end of each region, a final procedure is executed to store all the variables in $V_M(r_i)$ to their corresponding memory address. After doing this, a host-imported function is invoked, as shown in Fig. 6. The implementation of this function is host-specific. However, it is used to signal a pending migration request:

(a) If a migration request is pending, the execution traps. Doing so allows us to distinguish between a regular termination of a function and a migration-triggered termination. Note that the termination of a computation always happens after a checkpoint is completed, preventing an abnormal situation in which a computation has to resume from a partial checkpoint.
(b) Otherwise, the execution continues, marking the passage to region r_{i+1}.

These procedures are injected as Wasm bytecode: that choice implies that, on different hosts, the injected functions will be compiled to the proper target architecture with the rest of the Wasm module. Resuming the state of linear memories, however, requires more care. Issues may arise when restoring linear memory: consider, for example, the case of a workload function f_1 with checkpoints invoked within another function f_2, both accessing a linear memory m. Here, restoring m when the module is instantiated would result in an inconsistent state, where f_2 expects a blank memory, as is the case for any newly initialized linear memory, but an already initialized one is given instead. To prevent those inconsistencies, linear memories should be restored at the entrance of the resuming function, and not earlier. In the example

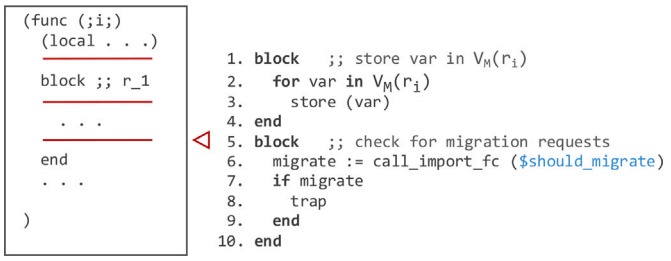


Fig. 6. The third runtime procedure, storing the variables in $V_M(r_i)$ in memory and then trapping in case a migration request is pending. Again, illustrating the mechanism of the procedure through pseudo-code instead of textual Wasm bytecode.

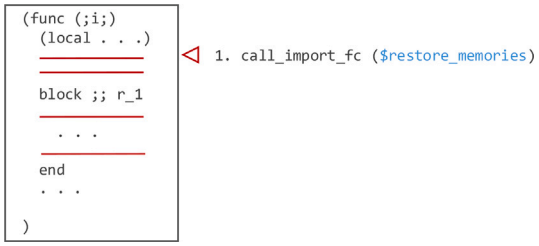


Fig. 7. An additional invocation to a host-imported function is required before starting the function body. `$restore_memories` should restore the content of any linear memories accessed in the module.

before, at the beginning of f_1. To do so, another host-defined imported function is used, as shown in Fig. 7.

Regarding the possibility of checkpointing multiple functions, it is sufficient to repeat the insertion procedure, provided a different function index. A dedicated linear memory will be reserved for each checkpointing function.

3.4. From distributed to centralized checkpoints

Performing regular checkpoint and restore procedures could be burdensome. Our experiments show that this might even double the execution time. While an IC system might be ready to pay such a toll for ensuring forward progress, it might be deemed excessive for applications in the Continuum. And while the distributed checkpoint strategy illustrated earlier is useful as a baseline, a more efficient strategy would definitely be more appealing for real-world scenarios. So, instead of *distributing* the checkpoints in each region, we developed a *centralized* approach. A centralized checkpoint is computed at most once during a function execution on a given node, and only when a migration request is pending. Conversely, resuming is performed as a single procedure at the beginning of a region after a migration, in a centralized manner. To do so, we have to slightly alter the three procedures discussed in Section 3.3, as discussed next. Moreover, the set of variables to be checkpointed could be further stretched to reduce the number of variables in a checkpoint. We adopt a similar strategy as in [41], which consists of selecting a subset of the modified variables in a region, which is also a subset of the *live-out* set of variables.[1] Given a region r_i, $V_{LO}(r_i)$ is the set of variables that are *live*, hence that are read, in any successor region of r_i. To compute the set V_{LO} of a region, the proposed algorithm uses the sets of *live-in* variables for all its [the region] successive regions. The live-in variables of a region r_i are the variables that are read within r_i. $V_{LI}(r_i)$ denotes their set. Finally, given $V_{M,0}(r_i)$ the set of modified variables from the beginning of the function

[1] Actually, the authors of [41] consider registers, and not variables, as elements of their live-out set.

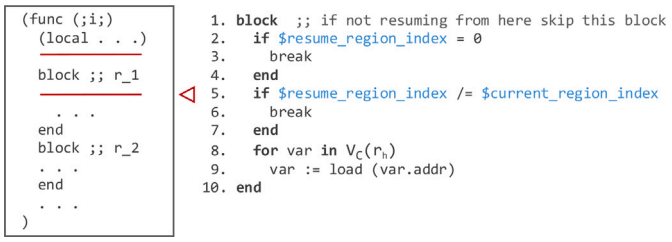


Fig. 8. The second runtime procedure for the centralized checkpoint strategy, in pseudo-code. Note that r_h is the direct predecessor of r_i.

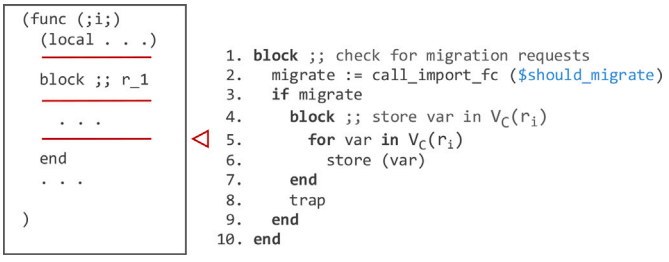


Fig. 9. The third runtime procedure for the centralized checkpoint strategy, in pseudo-code. The store operations are executed only in the presence of a migration request.

body up to the end of the current region r_i, the set of variables that are required to restore the state of the computation is:

$$V_C(r_i) = V_{M,0}(r_i) \cap V_{LO}(r_i) \tag{1}$$

$$= V_{M,0}(r_i) \cap \bigcup_{r_j \in next(r_i)} V_{LI}(r_j), \tag{2}$$

where $next(r_i)$ produces a set of regions coming after r_i. With respect to the distributed checkpoint strategy, the three injected runtime procedures undergo the following modifications:

1. The initial procedure, loading from memory the index of the region from which to resume, is identical to that in the distributed solution. Its graphical representation in Fig. 4 is still valid.
2. The behavior of the procedure at the entrance of a region r_i differs instead, as depicted in Fig. 8:
   (a) If the computation is resuming, but from a later region, the execution jumps to the end of the current region, *without* resuming any variables.
   (b) If the computation is resuming from this region r_i, then, given r_h the direct predecessor of r_i, the procedure restore from memory the variable in $V_C(r_h)$. Then the execution continues.
   (c) If the computation is not resuming, then the execution continues without performing any restore operation.
3. The checkpoint procedure performed at the end of a region, shown in Fig. 9, starts by checking whether there is any pending migration request:
   - If so, the execution stores in a dedicated linear memory the variables in $V_C(r_i)$. Then the computation traps as in the distributed case.
   - Otherwise, the execution continues, *without* storing any variables in memory.

In conclusion, to provide a qualitative comparison between the two solutions, we note that the distributed approach always invariably performs checkpoints at the end of each region. This implies that a computation that does not migrate during execution will still suffer from migration-related overhead. Conversely, in the centralized approach, a computation that does not migrate incurs a much smaller run-time penalty.

3.5. Implementation

Starting from `wasm-tools`, a toolset proffering parsing capabilities for WebAssembly bytecode, an additional tool, `wasm-migrate`, has been developed. This additional instrument elaborates a WebAssembly module to inject the runtime procedures previously described. In addition, it handles the insertion of two imported functions, thus re-indexing the local functions. In its current version, this tool receives as input a Wasm module and a function index. Moreover, it offers two algorithms for performing checkpoint and restore, one distributed, as discussed in Section 3.3, and one centralized as presented in Section 3.4. `wasm-migrate` is available as part of an open-source extension of `wasm-tools` at [42].

Injecting instructions within an existing Wasm module might hinder its validity, e.g., breaking the correct section order. To avoid this, instructions and blocks have to be inserted with respect to the Wasm specification. Within its toolset, `wasm-tools` offers a `validate` command to ensure that a module or component is actually well-formed.

The current implementation also aims to support Wasm components. This requires selecting the intended module within a component, knowing that components might contain more modules and can be nested within other components. This is already supported by our extension of `wasm-tools`. Additionally, it requires an integration with the specification language Wit, which is used to define the interfaces for interacting with a Wasm component, and is entirely absent in Wasm modules.[2]

Working exclusively with Wasm bytecode, thus ahead of execution, allows running the same module in *any* Wasm runtime, as long as the module imports and exports are satisfied. This choice makes this solution entirely agnostic of the underlying runtime.

3.6. Quality assessment

Assessing the quality of the proposed solution poses some challenges. The main one is that an exhaustive evaluation must consider the role of coordination, or orchestration, of the migrating computations. This falls outside the objective of this work, but we are actively working in this direction. In addition, at this time, we are not aware of any other solution supporting live migration for compiled Wams computations. Thus, we lack a comparison for reference. However, while acknowledging those limitations, an empirical assessment of the run-time penalty incurred while performing regular checkpoint and restore procedures could offer valuable insights. To this end, we selected an open-source benchmark and used it to measure the additional time taken to complete a computation while periodically creating checkpoints. Doing so allows us to estimate the run-time penalty introduced by the proposed mechanisms as a function of the number of checkpoints and the number of variables in $V_M(r_i)$ for each region r_i.

4. Initial empirical evaluation

The objective of this experiment is to measure the run-time penalty incurred while performing regular checkpoints. To do so, we have selected PolyBenchC [43], an open-source benchmark suite containing mathematical computations. Although other works have used this benchmark for the evaluation of Wasm-based computation, it has been criticized as unable to represent real-world Wasm components in [44,

[2] We are currently working to define an interface for managing the imports and exports envisioned by our checkpoint and restore strategy.

45]. Even if supporting this criticism, we regard this benchmark as sufficient to warrant some initial quantitative considerations on the proposed algorithms, as well as to provide a baseline reference for comparing different checkpoint strategies.

The execution of the experiment proceeds as follows. We compiled all the computations in PolyBenchC to Wasm (`wasi32`), then injected the runtime procedures for checkpoint and restore to the core function in each computation, namely the *kernel* functions performing the bulk of the benchmarks' workload. Finally, we measure the completion time for the original Wasm computations and those with checkpoint capabilities, either with a centralized or a distributed checkpoint strategy. Some tuning of the host environment is needed to warrant a fair comparison between the three instances. The experiments were performed on a Lenovo ThinkPad E15 computer running Ubuntu 22.04.01. To reduce the causes for variability, before running the experiments, hyperthreading was disabled, and each computation was bound to a specific core through an affinity attribute. Doing so prevents the Linux scheduler from scheduling a computation on another core after suspension. The chosen core on the hosting machine does not have any other core on its unit. Then, the scaling features of the selected core were disabled, as well as any deep-idle states, and its maximal and minimal allowed frequencies were fixed to the average one provided by the manufacturer. These steps are required to stabilize (or try to) the core working frequency. Finally, to reduce the chance of interference by other processes, each experiment has been launched running at the highest priority according to the Linux real-time scheduling FIFO policy. The details of this configuration are also available in our open-source repository [42]. It should be noted that, changing the hosting platform, those same configuration has to be adapted to the new environment, and might change significantly.

On a tuned-up machine, each computation is executed between 5 and 20 times, contingent on the order of execution. The sample size is admittedly small. However, deprived of the commodity features of modern OSs, from multi-threading to frequency scaling, our experimental infrastructure needed several hours for all the computations in the benchmark to execute once. Interestingly, experiments running with our restricted configuration gain stability. So much so that the variability between different executions is generally low; hence, in the plots shown in this section, we use just the average time as we deem it consistent enough not to affect our conclusions. Furthermore, the computations were executed in batches of different sizes, with the sole aim of preventing, upon an OS crash, some configuration from remaining untested. We also acknowledge that some experiments were run with some slight interference, usually due to the occasional activity of a web browser.

As shown in Fig. 10(a), the runtime penalty incurred while producing distributed checkpoints is significant. The figure shows the overhead as the ratio between the average measured execution time $\overline{T_C}$ of a computation with distributed checkpoints and the average measured execution time of the original computation $\overline{T}$, resulting in $\frac{\overline{T_C}}{\overline{T}}$. We see that the execution time of a migrating computation with distributed checkpoints often more than doubles compared to the original computation, with $\frac{\overline{T_C}}{\overline{T}} > 2$ and, occasionally, $\frac{\overline{T_C}}{\overline{T}} \approx 3$. A first observation is that our proposal adds numerous memory accesses in order to store used variables. The algorithm applied for region creation might also be contributing to unnecessary memory accesses. It is important to notice also the amount of variables stored in memory at each checkpoint (green line). The average number of variables is always above 30, with a peak close to 250. Reducing the checkpoint size, hence the average number of variables in $V_M(r_i)$, should also expectedly reduce the incurred overhead. Additionally, the overhead seems to increase with the average value of $V_M(r_i)$. Fig. 11 plots the ratio between the number of regions and the number of blocks in the input Wasm function. In general, it seems that a higher ratio could be associated with a smaller overhead and a smaller $V_M(r_i)$ size. In this experiment, the average ratio obtained is 0.20, yet with great variability. Smaller regions might result in a smaller average cardinality for $V_M(r_i)$, thus limiting the size of a single checkpoint. Though the correlation is not strict, it suggests that finer-grained regions could improve performance by limiting the state size.

Finally, Fig. 12 shows the rising overhead incurred while increasing the number of checkpoints and the average size of $V_M(r_i)$. The temporal reference for the overhead has been measured in a specific test environment and would differ significantly on a different set-up. However, the profile of the run-time cost should – expectedly – stay stable, and it does not account for additional delay caused by memory contention with other computations.

The case for centralized checkpoints is different. As expected, the runtime cost for computations that do not migrate is negligible, as shown in Fig. 10(b). This is a desirable property. As mentioned in Section 3.3, in our implementation, a centralized checkpoint is computed only when a migration request is pending.

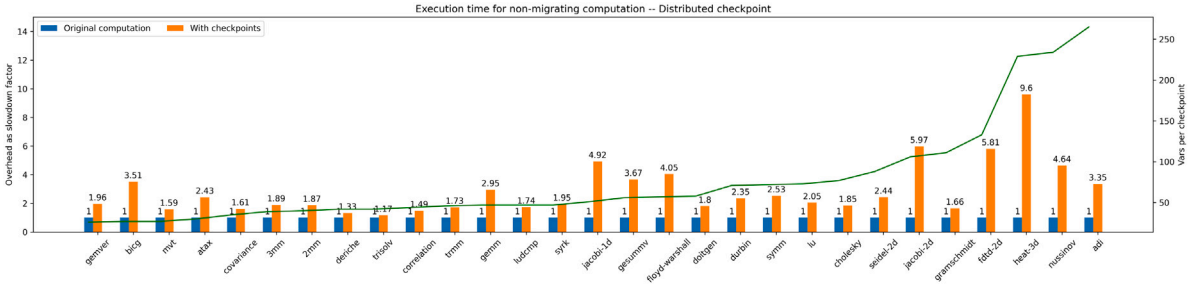
Finally, we propose an evaluation of the end-to-end execution time of a migration computation. To simulate a migration across different nodes, we developed a program that performs the following, sequential steps:

1. Initially, it set its affinity to a specific core, which, in our hosting machine, is core 8. Then it starts executing a Wasm computation in which the checkpoint and restore procedures have been injected, up to a certain point when a migration request is issued.
2. At this point, the computation traps and terminates, and the program takes care of saving its main linear memory, and its secondary checkpoint memory.
3. Then the computation changes its affinity, so that the program is moved to a different core, in our system core 4, which has the characteristic of not sharing its L2 cache with the previous core.
4. After moving to a different core, the Wasm computation resumes and terminates regularly.

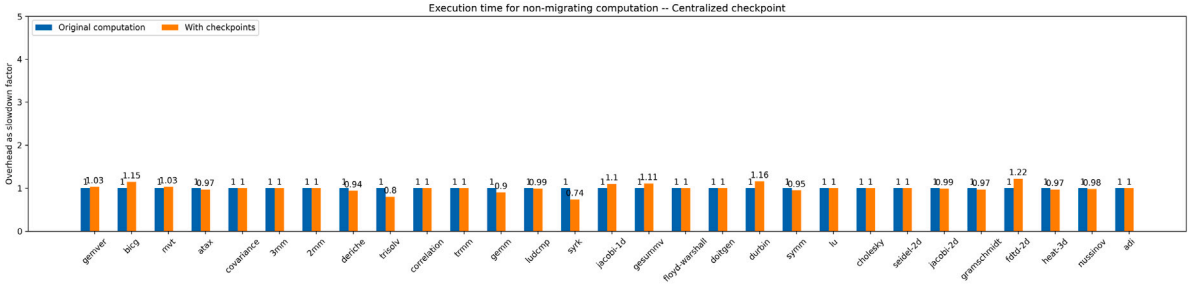
While performing this short procedure, we evaluated the execution time of some key procedures, as illustrated in Tables 5 and 6. To acquire those data, the rust library `rustix` was used, which provides APIs to the Linux system APIs, most notably for our purpose `clock_gettime` set with a monotonic clock. As shown in the two tables, the overhead associated with the checkpoint and restore procedures for the modules is below 30% of the same computation executed without migrating. In this case, the execution times dominate the overhead calculation, while the instantiation is comparatively smaller. We also note that restoring memory from the Wasm module incurs a significant overhead. Our implementation restores the *entire* linear memory. Future work might explore more selective and efficient approaches. In conclusion, compared to re-executing a computation, as during offloading, where the worst-case overhead might be 100% (full re-execution), migrating introduces a lesser overhead of nearly 30% to the computation.

Table 5
Measured times for performing a set of operations connected to the execution of a migrating Wasm module (3mm from PolyBenchC), in microseconds. The overhead associated with the checkpoint and restore procedures is less than 30% of the execution time of the same non-migrating computation.

Computation 3mm	
pre-instantiation (before migration)	220,014
instantiation (before migration)	556,219
execution time (before migration)	41.624.420,119
main memory copy (from host)	1.838.899,764
main memory copy (from Wasm)	6.889.289,298
pre-instantiation (after migration)	203,495
instantiation (after migration)	539,279
execution time (after migration)	7.565.491,427
execution time (without migration)	43.789.680,493



(a) The run-time cost of performing regular checkpoints. The Wasm computations with checkpoint capabilities are shown as orange bars, while their Wasm counterparts without checkpoints are shown as blue bars. The green line represents the average number of variables in $V_M(r_i)$ for each computation.



(b) The execution times for non-migrating computations with centralized checkpoints. The overhead is negligible. Occasionally, the execution time is lower for the migratable computation than its non-migrating counterpart, but this is due to the small dataset.

Fig. 10. Comparing the execution time of non-migrating computations, with and without checkpoints.

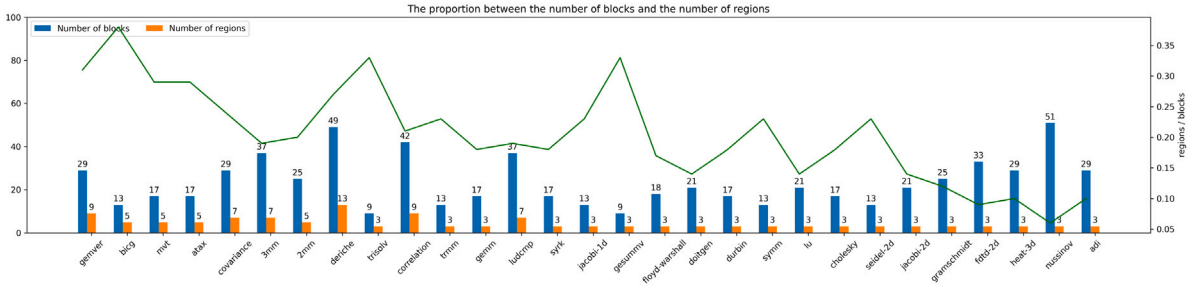


Fig. 11. The ratio between the number of regions and the number of blocks in the computation. The green line shows its value, which, on average, is 0.20. It seems that a higher ratio corresponds, associated with a smaller $V_M(r_i)$ size, to a smaller overhead.

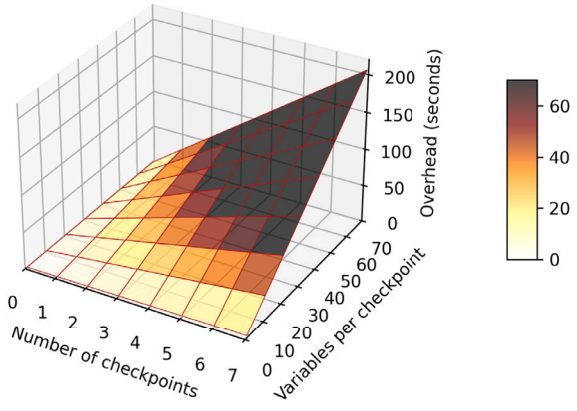


Fig. 12. Expected overhead with increasing number of checkpoints an average $V_M(r_i)$ size.

Table 6
Measured times connected to the procedures for migrating the computation correlation, in PolyBenchC, expressed in microseconds. Again, the overhead associated with the checkpoint and restore procedures is less than 30% of the execution time of the same non-migrating computation.

Computation correlation	
pre-instantiation (before migration)	214,138
instantiation (before migration)	550,956
execution time (before migration)	32.316.156,778
main memory copy (from host)	110.286,119
main memory copy (from Wasm)	4.561.384,095
pre-instantiation (after migration)	197,628
instantiation (after migration)	544,941
execution time (after migration)	5.012.925,895
execution time (without migration)	32.287.927,020

These measurements confirm that performing checkpoints might impose a high run-time penalty on the migrating computation if using a distributed checkpoint strategy as the one presented in Section 3.3. However, the same cost becomes negligible for non-migrating computations using the centralized approach presented in Section 3.4. Still, there is room for improvement in the region formation algorithm. However, the end-to-end cost of migrating a computation, at least in the two evaluated instances, is low enough to make this approach valuable for real-world applications. This motivates the enhancements discussed next. Moreover, it establishes a baseline for compiled Wasm migration

overhead, against which future optimizations can be measured and compared.

5. Discussion

In this work, a mechanism to perform checkpoint and restore of compiled Wasm computations is presented, and a preliminary evaluation of the possible run-time penalty incurred in using it, assessed. The live migration of components represents a pivotal element in our envisioned model, which, at its core, aims to proffer cyber–physical systems with that elasticity and reconfigurability that the Cloud world has so effectively made its own. If physical computing devices move in space, then software computations have to migrate too in order to react, or act in advance, of the shifting physical worlds. Technologies like Wasm and Rust, already granting some form of spatial isolation, enable migrating computations to co-exist with traditional embedded software, running and migrating on the resources unused by those latter computations.

As mentioned in the previous sections, three critical points emerge, which we discuss below in isolation. The first limitation concerns the algorithm used to partition a function into regions. Although fully automatic, it might be too coarse-grained for real-world applications. Two of these possible extensions are worth further investigation. A first approach would be to implement a similar mechanism to the one in [41]. In this work, the authors divide the program into regions bounded by a maximum duration, eventually performing loop-unrolling operations to achieve the desired region size. A similar algorithm can be used for Wasm computations. The second approach instead is to dispose of the requirement for autonomy in the creation of regions. In other words, consider the case of applications designed and developed to perform well, i.e., efficiently or with small overhead, in the presence of checkpoints. For instance, it would be interesting to consider the case of Wasm computations partitioned into steps, bounded by dependency constraints, in which migration might occur at the end of each step. Previous works in the EHS domain have already explored similar directions, for example, as in [40]. Also, for time-predictable applications, this might not be an entirely alien scenario.

The second concern arises from the partiality of an evaluation that does not consider the role of orchestration. In the envisioned model for the Continuum, two kinds of orchestration are needed. One that is *infrastructural*, which deals with the aggregation and assignment of resources to applications. The other happens *within* the application, to take care of making deployment and scheduling decisions about individual components. In this formulation, no industrial orchestrator that we know of suffices. Therefore, for the objective of this investigation, namely to investigate runtime facilities enabling live migration, we focused exclusively on the runtime machinery.

The third critical point is the lack of a representative benchmark for Wasm computation. Previous works have argued that PolyBenchC produces exceedingly positive results when applied to Wasm computation. Therefore, assessing the runtime cost of the proposed checkpoint and restore mechanism (or an upgraded version of it) over a more realistic benchmark is part of our future work. Using an open-source benchmark also earns clear benefits in terms of replicability. Even if not representative of real-world workloads, the measurements taken from this benchmark are more than enough to compare two checkpointing strategies and draw conclusions about them.

The proposed solution could also be extended to support, among other improvements, a mechanism for stack migration. WebAssembly is based on a stack machine, where instructions manipulate an *operand stack*, adding or removing values, or altering the control flow of the execution in a *structured* manner, which means well-nested (e.g., jumps occur outward, from a nested block to one of its parent blocks). In the current implementation, checkpoints occur when the operand stack is empty (in particular, between regions). Reasoning on a mechanism to store the operand stack of a function would earn more flexibility during region creation.

Notably, the proposed approach to live migration could be applied for Wasi-compliant components. Existing works, such as [33], show that embedded software in industrial scenarios can benefit from the decomposition of applications into Wasm components that interact through a standardized system interface (Wasi). Arguably, therefore, adhering to this interface standard is key to enable Wasm-based applications to interact with embedded devices.

6. Conclusion

In this work, we have proposed an original mechanism to perform checkpoint and restore of Wasm computations, inspired by a state-of-the-art approach coming from the domain of intermittent computing (IC), and applicable to any Wasm runtime. To our current knowledge, this is the first mechanism to support live migration of *compiled* Wasm modules across heterogeneous nodes, with heterogeneous Wasm runtimes. And the support for Wasm components is soon to follow. The proposed solution autonomously partitions a computation into regions and injects the run-time procedures required to save the current advance in the execution and terminate the computation in the presence of a pending migration. An empirical evaluation of two algorithms that implement this approach, based on the PolyBenchC suite, shows that the time to completion of the computations enriched with the checkpoint and restore procedures varies significantly depending on the algorithm. A distributed checkpoint approach might take on average twice the time of its non-migrating counterpart, due to the overhead of saving the computation state. In contrast, the centralized algorithm incurs negligible overhead.

By enabling portable, heterogeneous live migration at the WebAssembly level, this work takes a step toward a more dynamic and resilient compute continuum.

Future work will consider extending the current partitioning algorithm to create regions according to the already existing algorithm applied in the EHS domain. Another research direction stemming from the results of this work consists of investigating the end-to-end response time of a system where applications are intended as orchestrated aggregates of migrating computations. Notably, this implies reconsidering the functioning of orchestration, distinguishing between infrastructure-level and application-level orchestration.

CRediT authorship contribution statement

E. Tinto: Writing – original draft, Visualization, Software, Methodology, Investigation, Data curation, Conceptualization. **L. Marchiori:** Visualization, Software, Investigation, Data curation, Conceptualization. **T. Vardanega:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Conceptualization.

Funding sources

The work carried out in this project was partially funded under the National Recovery and Resilience Plan (NRRP), Mission 4 Component 2 Investment 1.4 - Call for tender No. 3138, 16 December 2021, by the Italian Ministry of University and Research funded by the European Union – NextGenerationEU.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] J. Ménétrey, M. Pasin, P. Felber, V. Schiavoni, WebAssembly as a common layer for the cloud-edge continuum, in: Proceedings of the 2nd Workshop on Flexible Resource and Application Management on the Edge, FRAME '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 3–8, http://dx.doi.org/10.1145/3526059.3533618.

[2] M. Jansen, A. Al-Dulaimy, A.V. Papadopoulos, A. Trivedi, A. Iosup, The SPEC-RG reference architecture for the compute continuum, in: 2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing, CCGrid, 2023, pp. 469–484, http://dx.doi.org/10.1109/CCGrid57682.2023.00051.

[3] A. Al-Dulaimy, M. Jansen, B. Johansson, A. Trivedi, A. Iosup, M. Ashjaei, A. Galletta, D. Kimovski, R. Prodan, K. Tserpes, G. Kousiouris, C. Giannakos, I. Brandic, N. Ali, A.B. Bondi, A.V. Papadopoulos, The computing continuum: From IoT to the cloud, Internet Things 27 (2024) 101272, http://dx.doi.org/10.1016/j.iot.2024.101272, URL https://www.sciencedirect.com/science/article/pii/S2542660524002130.

[4] L. Coppolino, S. D'Antonio, G. Mazzeo, R. Nardone, L. Romano, M. Schmitt, WASMBOX: A lightweight wasm-based runtime for trustworthy multi-tenant embedded systems, IEEE Trans. Emerg. Top. Comput. (2024) 1–14, http://dx.doi.org/10.1109/TETC.2024.3409817.

[5] S. Wallentowitz, B. Kersting, D.M. Dumitriu, Potential of WebAssembly for embedded systems, in: 2022 11th Mediterranean Conference on Embedded Computing, MECO, 2022, pp. 1–4, http://dx.doi.org/10.1109/MECO55406.2022.9797106.

[6] I. Zyrianoff, L. Sciullo, L. Gigli, A. Trotta, C. Kamienski, M. Di Felice, An over the air software update system for IoT microcontrollers based on WebAssembly, in: 2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things, DCOSS-IoT, 2024, pp. 331–338, http://dx.doi.org/10.1109/DCOSS-IoT61029.2024.00057.

[7] E. Wen, G. Weber, Wasmachine: Bring the edge up to speed with a WebAssembly OS, in: 2020 IEEE 13th International Conference on Cloud Computing, CLOUD, 2020, pp. 353–360, http://dx.doi.org/10.1109/CLOUD49709.2020.00056.

[8] L. Seidel, J. Beier, Bringing rust to safety-critical systems in space, in: 2024 Security for Space Systems, 3S, 2024, pp. 1–8, http://dx.doi.org/10.23919/3S60530.2024.10592287.

[9] A. Pinho, L. Couto, J. Oliveira, Towards rust for critical systems, in: 2019 IEEE International Symposium on Software Reliability Engineering Workshops, ISSREW, 2019, pp. 19–24, http://dx.doi.org/10.1109/ISSREW.2019.00036.

[10] Ferrocene, Ferrocene, 2024, https://ferrous-systems.com/ferrocene/.

[11] wasmtime, Wasmtime, 2024, https://wasmtime.dev/.

[12] CRIU, Checkpoint/restore in userspace, 2025, https://criu.org/Main_Page.

[13] S. Nadgowda, S. Suneja, N. Bila, C. Isci, Voyager: Complete container state migration, in: 2017 IEEE 37th International Conference on Distributed Computing Systems, ICDCS, 2017, pp. 2137–2142, http://dx.doi.org/10.1109/ICDCS.2017.91.

[14] T. Xing, A. Barbalace, P. Olivier, M.L. Karaoui, W. Wang, B. Ravindran, H-container: Enabling heterogeneous-ISA container migration in edge computing, ACM Trans. Comput. Syst. 39 (1–4) (2022) http://dx.doi.org/10.1145/3524452.

[15] K. Govindaraj, A. Artemenko, Container live migration for latency critical industrial applications on edge computing, ETFA, in: 2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation, vol. 1, 2018, pp. 83–90, http://dx.doi.org/10.1109/ETFA.2018.8502659.

[16] B. Xu, S. Wu, J. Xiao, H. Jin, Y. Zhang, G. Shi, T. Lin, J. Rao, L. Yi, J. Jiang, Sledge: Towards efficient live migration of docker containers, in: 2020 IEEE 13th International Conference on Cloud Computing, CLOUD, 2020, pp. 321–328, http://dx.doi.org/10.1109/CLOUD49709.2020.00052.

[17] D. Fujii, K. Matsubara, Y. Nakata, Stateful VM migration among heterogeneous WebAssembly runtimes for efficient edge-cloud collaborations, in: Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking, EdgeSys '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 19–24, http://dx.doi.org/10.1145/3642968.3654816.

[18] H.-J. Jeong, C.H. Shin, K.Y. Shin, H.-J. Lee, S.-M. Moon, Seamless offloading of web app computations from mobile device to edge clouds via HTML5 web worker migration, in: Proceedings of the ACM Symposium on Cloud Computing, SoCC '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 38–49, http://dx.doi.org/10.1145/3357223.3362735.

[19] M. Nurul-Hoque, K.A. Harras, Nomad: Cross-platform computational offloading and migration in femtoclouds using WebAssembly, in: 2021 IEEE International Conference on Cloud Engineering, IC2E, 2021, pp. 168–178, http://dx.doi.org/10.1109/IC2E52221.2021.00032.

[20] D. Gohman, L. Clark, A. Crichton, A. Brown, S. Clegg, P. Hickey, Yosh, D. Bakker, C. Ihrig, Y. Yuji, P. Huene, J. Konka, P. Sikora, C. Dickinson, M. Frysinger, R. Brown, Y. Takashi, S. Akbary, S. Rubanov, M. Berger, J. Triplett, G. Kulakowski, E. Crosson, D. Vasilik, B. Hayes, A. Turner, M. Christian, MaulingMonkey, M. Sebrechts, N. McCallum, WebAssembly/WASI: v0.2.2, Zenodo, 2024, http://dx.doi.org/10.5281/zenodo.13888155.

[21] V.A.B. Pop, A. Niemi, V. Manea, A. Rusanen, J.-E. Ekberg, Towards securely migrating webassembly enclaves, in: Proceedings of the 15th European Workshop on Systems Security, EuroSec '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 43–49, http://dx.doi.org/10.1145/3517208.3523755.

[22] M. Kreutzer, M. Seidler, K. Dudzik, V.P. Betancourt, J. Becker, Migration of isolated application across heterogeneous edge systems, in: 2024 IEEE 8th International Conference on Fog and Edge Computing, ICFEC, 2024, pp. 64–70, http://dx.doi.org/10.1109/ICFEC61590.2024.00014.

[23] R. Liu, L. Garcia, M. Srivastava, Aerogel: Lightweight access control framework for WebAssembly-based bare-metal IoT devices, in: 2021 IEEE/ACM Symposium on Edge Computing, SEC, 2021, pp. 94–105, http://dx.doi.org/10.1145/3453142.3491282.

[24] Y. Yang, A. Hu, Y. Zheng, B. Zhao, X. Zhang, A. Quinn, Migratable velocity virtual machine, 2024, URL https://arxiv.org/abs/2410.15894v1.

[25] Wanco, 2025, URL https://github.com/tamaroning/wanco.

[26] S.W. Ali, Z. Tong, J. Goh, J.H. Anderson, Predictable GPU Sharing in Component-Based Real-Time Systems (Artifact), in: S.W. Ali, Z. Tong, J. Goh, J.H. Anderson (Eds.), Dagstuhl Artifacts Ser. 10 (1) (2024) 1:1–1:5, http://dx.doi.org/10.4230/DARTS.10.1.1, URL https://drops.dagstuhl.de/entities/document/10.4230/DARTS.10.1.1.

[27] J. Bakita, J.H. Anderson, Hardware compute partitioning on NVIDIA GPUs, in: 2023 IEEE 29th Real-Time and Embedded Technology and Applications Symposium, RTAS, 2023, pp. 54–66, http://dx.doi.org/10.1109/RTAS58335.2023.00012.

[28] P. Burgio, M. Bertogna, I.S. Olmedo, P. Gai, A. Marongiu, M. Sojka, A software stack for next-generation automotive systems on many-core heterogeneous platforms, in: 2016 Euromicro Conference on Digital System Design, DSD, 2016, pp. 55–59, http://dx.doi.org/10.1109/DSD.2016.84.

[29] WebAssembly, WebAssembly, WebAssembly Community Group, 2022, https://webassembly.github.io/spec/core/index.html.

[30] S. Klabnik, Is WebAssembly the return of java applets & flash?, 2018, https://steveklabnik.com/writing/is-webassembly-the-return-of-java-applets-flash.

[31] N. Fitzgerald, Security and correctness in wasmtime, 2022, https://bytecodealliance.org/articles/security-and-correctness-in-wasmtime.

[32] H. Xu, F. Kjolstad, Copy-and-patch compilation: a fast compilation algorithm for high-level languages and bytecode, Proc. ACM Program. Lang. 5 (OOPSLA) (2021) http://dx.doi.org/10.1145/3485513.

[33] M.V. Kenhove, M. Seidler, F. Vandenberghe, W. Dujardin, W. Hennen, A. Vogel, M. Sebrechts, T. Goethals, F.D. Turck, B. Volckaert, Cyber-physical WebAssembly: Secure hardware interfaces and pluggable drivers, 2025, URL https://arxiv.org/abs/2410.22919v2, arXiv:2410.22919.

[34] M. Seidler, A. Krause, P. Ulbrich, Extending lifetime of embedded systems by WebAssembly-based functional extensions including drivers, 2025, URL https://arxiv.org/abs/2503.07553v1, arXiv:2503.07553.

[35] G. Bartolomeo, M. Yosofie, S. Bäurle, O. Haluszczynski, N. Mohan, J. Ott, Oakestra: A lightweight hierarchical orchestration framework for edge computing, in: 2023 USENIX Annual Technical Conference, USENIX ATC 23, USENIX Association, Boston, MA, 2023, pp. 215–231, URL https://www.usenix.org/conference/atc23/presentation/bartolomeo.

[36] S. Kakati, M. Brorsson, A cross-architecture evaluation of WebAssembly in the cloud-edge continuum, in: 2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing, CCGrid, 2024, pp. 337–346, http://dx.doi.org/10.1109/CCGrid59990.2024.00046.

[37] L. Poggiani, C. Puliafito, A. Virdis, E. Mingozzi, Live migration of multi-container kubernetes pods in multi-cluster serverless edge systems, in: Proceedings of the 1st Workshop on Serverless At the Edge, SEATED '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 9–16, http://dx.doi.org/10.1145/3660319.3660330.

[38] wasm portability, Portability, W3C Community Group, 2024, https://webassembly.org/docs/portability/.

[39] M. Surbatovich, B. Lucia, L. Jia, Towards a formal foundation of intermittent computing, Proc. ACM Program. Lang. 4 (OOPSLA) (2020) http://dx.doi.org/10.1145/3428231.

[40] K. Maeng, A. Colin, B. Lucia, Alpaca: intermittent execution without checkpoints, Proc. ACM Program. Lang. 1 (OOPSLA) (2017) http://dx.doi.org/10.1145/3133920.

[41] J. Choi, L. Kittinger, Q. Liu, C. Jung, Compiler-directed high-performance intermittent computation with power failure immunity, in: 2022 IEEE 28th Real-Time and Embedded Technology and Applications Symposium, RTAS, 2022, pp. 40–54, http://dx.doi.org/10.1109/RTAS54340.2022.00012.

[42] E. Tinto, Wasm-Tool (Extended with Wasm-Migrate), University of Padova, 2025, https://github.com/TintoEdoardo/wasm-tools/tree/develop.

[43] polybenchc, PolyBenchC the polyhedral benchmark suite, 2024, https://www.cs.colostate.edu/~pouchet/software/polybench/.

[44] A. Jangda, B. Powers, E.D. Berger, A. Guha, Not so fast: Analyzing the performance of WebAssembly vs. Native code, in: 2019 USENIX Annual Technical Conference, USENIX ATC 19, USENIX Association, Renton, WA, 2019, pp. 107–120, URL https://www.usenix.org/conference/atc19/presentation/jangda.

[45] D. Baek, J. Getz, Y. Sim, D. Lehmann, B.L. Titzer, S. Ryu, M. Pradel, Wasm-R3: Record-reduce-replay for realistic and standalone WebAssembly benchmarks, Proc. ACM Program. Lang. 8 (OOPSLA2) (2024) http://dx.doi.org/10.1145/3689787.